Can the thesis be made simpler?

What stays, version 3

For review before anything is applied

16 October 2026

Nothing in the thesis or the code has been changed. The controller redesign, with a real centralised and a real section-based controller, is in `real_centralised_design.md`. This note keeps only what is independent of it, which is where the complexity sits, the memory change of issue #160, and the factor-grid lever.

1 Where the complexity actually sits

All counts are measured on the current chapters with `texcount`, about 18 300 words in `2.Introduction.tex`, `3.Background.tex` and `4.Implementation.tex`.

- **The experimental design.** The aim and scope section lists five independent variables, which are storage mode, architecture, congestion sensitivity, communication, and load. Crossed in full with three storage rules and four routing variants this is up to 24 conditions before load. I did not check which combinations the thesis really plans to run.
- **Open parameters.** The Implementation chapter has 19 cells marked TBD.
- **Change history.** The two chapters carry 39 distinct `[Axx]` tags, and `GAPS.md` holds 30 entries, numbered up to `[A31]`, over about 1 000 lines.
- **Notation.** The notation table has about 106 rows.

Text edits alone move these little. A passage the size of the halo paragraph is about 155 words, under 1% of the text. The larger levers are the design of the experiments and the open parameters.

2 Embed each window once

This is the design in `embed_once_design.md`, now issue #160. It is a memory and scaling change and does not simplify the thesis. The text already says the forward pass takes all agents of a timestep together and that a message is the sender's embedding, so no sentence changes. It applies to the decentralised arm with communication, the only arm with senders. In the code the net effect is roughly neutral. It removes the six sender window keys and their padding, and it adds keys, a lookup, a two-pass value computation and a minibatch iterator built from whole timesteps. The message cap `max_messages` appears 98 times across 16 files and could become the fleet size.

3 The factor-grid lever

You agreed with this. Run the main comparison of storage rule against architecture at one congestion setting and one load, and treat congestion sensitivity and load as separate one-factor sweeps.

- The main grid is $3 \times 3 = 9$ conditions.
- Fully crossed with load at 3 levels (my assumption, the thesis says several) the design is $24 \times 3 = 72$ conditions. With sweeps it is about 18 to 24, depending on how many storage rules the load sweep repeats.

The limit. RQ4 asks how the *effects* change with load. The effect of coupling is adaptive against fixed storage, and it differs by architecture, so the load sweep must repeat that comparison for every architecture. It cannot run at one storage rule. RQ3 needs congestion sensitivity switched on and off in both the storage and the routing layer, so the congestion sweep needs the routing toggle for each architecture as well. One-factor sweeps also lose interactions, such as whether congestion sensitivity matters more under high load.

Verdict. The largest single saving in conditions, and in the text that explains them. It is a change of design, and a conversation with the supervisor is worth having.

4 What I have not checked

- How many conditions the thesis plans to run, and whether congestion sensitivity applies to every storage rule.
- The probe for issue #160, which tests that a learner subclass works with the single local learner here.